Tree

沈威宇

November 1, 2026

1 Tree

I Tree

i Tree

A tree is an abstract data type of a set of nodes. A relationship between two nodes i, j is defined with

- i is called a child of j and
- j is called a parent of i; and
- a node can have at most one parent.

If not empty, a tree is recursively defined as a distinguished node r called root and zero or more nonempty trees called subtrees, where the root of each substree is a child of r.

- Nodes sharing the same parent are called siblings. Children of a node are defined with a left to right order.
- A node with no children is called a leaf, external node, or exterior node.
- A node that is not a leaf is called an internal node or interior node.
- The number of edges from the root to a node is called the depth of the node, while the root has depth 0.
- The maximum depth of the nodes is called the height or depth of the tree.
- The ancestors of a node is recursively defined as itself for root and itself and the ancestors of its parent for non-root nodes.
- The descendants of a node is recursively defined as itself for leaves and itself and the descendants of its children for internal nodes.
- Nodes of the same depth are called a level. Levels are ordered by depth. The left-to-right order of nodes in a level is defined recursively to, if they are siblings, follow that of siblings; otherwise follow that of their parents.
- A tree consisting of all descendants of a node is called a subtree. The left-to-right order of subtrees whose roots are of the same depth are defined as that of their roots.

ii Tree representation

A representation of tree where each pair of parent and child is connected with a edge.

iii Left child, right sibling representation

A representation of tree where each internal node and its leftmost child is connected with a edge, and each sibling that is not the leftmost one is called to the sibling at its left.

II n-ary tree

i n-ary tree

A tree in which no node can have more than n children is called an n-ary tree.

ii Full, proper, or plane

An n-ary tree where each internal node has exactly n children is called full, proper, or plane.

iii Filled

A level of depth $k \in \mathbb{N}$ of an n-ary tree is called filled iff each node in the level of depth $k-1$ has exactly n children. The level of depth 0 is called filled iff the tree is nonempty.

iv Complete

An n-ary tree where every level, except possibly the last, is filled and whose nodes in the last level are as far left as possible, is called complete.

A complete n-ary tree is indexed from top to bottom and then from left to right, with the root as node 1.

v Perfect

A complete n-ary tree where all leaves are of the same depth is called perfect.

A perfect n-ary tree with height k has $\sum_{j=0}^{k} n^j$ nodes.

III Binary tree

i Binary tree

In a binary tree, the two possible children of a node are called left child and right child. A child is either left or right but not both even if it has no sibling.

ii Complete binary tree

For a complete binary tree with n nodes:

- Parent of node i is node $\left\lfloor \frac{i}{2} \right\rfloor$ for all integers $i > 1$.
- The left child of node i is $2i+1$ for all integers $i \leq \frac{n-1}{2}$.
- The right child of node i is $2i$ for all integers $i \leq \frac{n}{2}$.

iii Perfect binary tree

A perfect binary tree with height k has $2^{k+1}-1$ nodes.

iv Balanced tree

A type of binary tree with height $O(\log n)$ where n is the number of nodes is called balanced.

IV Tree traversal

They are defined recursively. Suppose the root is r.

i Preorder

First visit r, and then visit the subtrees whose roots are children of r from left to right.

ii Postorder

Visit the subtrees whose roots are children of r from left to right, and finally visit r.

iii Inorder (binary tree)

Among the subtrees whose roots are children of r, first visit the left subtree, and then visit r, and then visit the right subtree.

V Binary search tree (BST)

i Binary search tree (BST)

Stores keys in the nodes of a binary tree so that searching, insertion and deletion can be done efficiently. For every node x, all the keys in its left subtree are smaller than x and all keys in its right subtree are larger than x.

ii Searching

Suppose we are searching for x and the root is r.

- If $x = r$, then we are done.
- If $x < r$, then we search in the left subtree.
- If $x > r$, then we search in the right subtree.

The time complexity is $O(\text{height of tree})$.

iii Finding minimum and maximum

To find the node containing the smallest key, start at root and go to the left child as long as there is one, and the leaf we stop at is the smallest key. Similarly for find the largest key.

The time complexity is $O(\text{height of tree})$.

iv Insertion

Suppose we want to insert x. First search for it, if we found it, do noting (or update something); otherwise, if we stop at y,

- if $x > y$, insert x as the right child of y if $x > y$, and
- if $x < y$, insert x as the left child of y.

The time complexity is $O(\text{height of tree})$.

v Deletion

Suppose we want to delete x. First search for it, if we don't found it, do noting (or update something); otherwise,

- if it is a leaf, delete it,
- if it has one child y, adjust y as a child of the parent of x of the same position as x and delete x, and
- if it has two children, replace it with the minimum element y in the subtree whose root is the right child of x and delete node y, which is a leaf.

The time complexity is $O(\text{height of tree})$.

VI AVL tree (Adelson-Velsky and Landis tree)

A